

L1 – Introduction & Task Decomposition

Vocabulary & Platform Model

Compute Hierarchy

Host **delegates to** a Compute Device.

- **Host**: controller that delegates
- **Device**: receives the work
- **CU** → **PE** → **FU** (units inside the device)
- **UE**: process/thread running a task

Task

- OS sense: a process **or** a thread
- parallel sense: a piece of a parallel program (a unit of work)

Recipe: Map Platform → Terminology

Find the host

which side delegates work.

Find the device

which side receives it; inside it CU → PE → FU.

Identify UEs

each spawned process/thread; a **task** is one work slice it runs.

Ex. S12/13 – CB1 / Jetson terminology

Programming on the CB1, delegating to the Jetson Nano. What is what?

CB1 = **Host**; Nano = **Device**; Nano SMs = CUs; CUDA cores = PEs; ALU/FPU = FU; spawned threads = UEs; one Sobel slice = task.

Parallel Patterns

Pattern Hierarchy

Worked top→bottom:

1. **Finding Concurrency** – expose parallelism
2. **Algorithm Structure** – organise tasks
3. **Supporting Structure** – code/data structures
4. **Implementation Mechanisms** – threads, sync

Finding Concurrency

Two groups of patterns:

- **Decomposition**: split by **function** (task) or by **data/loop**
- **Dependency Analysis**: Group / Order / Data-sharing

Ex. S22/23/25 – Name the patterns

State the hierarchy and the Finding-Concurrency patterns.

Finding Concurrency → Algorithm Structure → Supporting Structure → Implementation Mechanisms; Finding Concurrency = **Decomposition** (task/data) + **Dependency Analysis** (group/order/share).

The Sobel Use Case

Sobel Pipeline (6 tasks)

read_image → rgb_2_grey → {sobel_x || sobel_y} → sobel_all → display_image

- only sobel_x & sobel_y are independent

Sobel Operator

$$G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$$

$$|G| = \sqrt{G_x^2 + G_y^2} \rightarrow |G_x| + |G_y| \text{ (simplified)}$$

Ex. S18 – How does Sobel work?

Determine the edge-detection computation.

Two 3×3 convolutions give the gradient G_x, G_y ; magnitude $|G|$ **simplified** to $|G_x| + |G_y|$, then threshold.

Task Decomposition

Functional vs Loop Splitting

- **Functional**: split by named function (few tasks, limited)
- **Loop/data**: split per-element loop by region (scales – preferred)

CFG / DFG

Control-Flow Graph: all execution paths (nodes = actions, edges = order). **Data-Flow Graph**: data dependencies. Used to spot concurrency.

Recipe: Task-Decompose an Algorithm

Draw the CFG

nodes = functions, edges = data deps.

Count distinct actions

= functional tasks.

Find independent tasks

no path between ⇒ concurrent.

Try both splittings

functional vs loop/region.

State parallelism + joins

Ex. S26–29 – Decompose Sobel / read CFG

Split Sobel into tasks. Distinct actions? Independent tasks? Parallelisation potential?

6 functional tasks; **6** distinct actions; only sobel_x, sobel_y independent. Functional parallelism limited (2-way); **loop/region splitting** scales.

Dependency Analysis

Group / Order / Share

- **Group**: tasks with same data/constraints
- **Order**: producer before consumer
- **Data-sharing**: classify each datum

Data-Sharing Types

- read-only
- effectively-local
- read-write (accumulate / multi-read-single-write)

Recipe: Dependency Analysis

Group

same data/constraints together.

Order

add constraint per producer→consumer (+ ghost-exchange before boundary).

Classify

read-only / local / read-write; strict ordering removes “potentially”.

Ex. S30–33 – Apply to Sobel

Which sub-pattern matters most? Group/order/share for Sobel?

Data Sharing. Group `sobel_x/y` (same grey input); order `sobel_all` after both; grey = read-only, kernel = effectively-local, `Gx/Gy` = read-write, `sobel_all` = **accumulate**.

Task Parallelism & Fork–Join

Task-Parallelism Checks

- compute > management overhead
- $\#tasks \geq \#PEs$ (more is better)
- minimise data dependencies
- equal work → static; uneven → dynamic / work-stealing

Fork–Join

Clone code into concurrent tasks, then **join** (wait). **Not** POSIX `fork()`; suits recursive/irregular work.

Recipe: Evaluate a Parallel Design

Check compute-vs-overhead, $\#tasks$ -vs- $\#PEs$, dependencies, schedule choice.

Ex. S34–39 – Evaluate & fork-join

Compute vs management? Tasks vs PEs? Is fork-join suitable for Sobel?

Whole-function task justifies a UE (per-pixel doesn't); want $\#tasks \geq \#PEs \rightarrow$ region-split; only dep = `sobel_all` join; static schedule. Fork-join: **poor fit** for a fixed pipeline (it suits irregular/recursive work).

Processes & Threads

Process

Running program.

- unit of resource + scheduling
- own `main()` / address space
- MPU-isolated; `fork()` = CoW clone

Thread

Inside a process.

- shares the process's memory
- cheap → reuse via **thread pool**

Ex. S41–45 – Process vs thread

What is a process? How do they differ from threads?

Process = isolated running program (own address space, `fork()`/`wait()`). Thread = lightweight, shares memory, created with `pthread_create`; expensive to create → pool.

L2 – Data Decomposition

Data Decomposition

Data Decomposition

Split the **data structure** (image array) into chunks updated independently. Scales with $\#$ chunks.

Granularity

- big chunk: low parallelism, low overhead
- small chunk: high parallelism, high overhead + all-boundary
- largest = whole image; smallest = 1 pixel

Recipe: Choose & Size Data Chunks

Feasible?

heavy work = data-structure manipulation + similar op applied independently?

Pattern

array + local neighbourhood $\rightarrow$ Geometric Decomposition.

Shape

minimise surface/volume $\rightarrow$ square tiles.

Boundaries

1-cell ghost halo; exchange a-priori or overlapped.

Map

$\#$ chunks $>$ $\#$ UEs for balancing.

Ex. S6–8 – Feasible for edge detection?

Is data decomposition feasible? Tasks? Granularity boundaries?

Yes: Sobel over every pixel, kernel needs only a 3×3 neighbourhood $\rightarrow$ regions independent. Tasks = per-region filters. Whole image = no parallelism; 1 pixel = overhead dominates.

Geometric Decomposition

Geometric Decomposition

Array problems with local-neighbourhood (stencil) operations $\rightarrow$ split into tiles. Interior = embarrassingly parallel; only boundaries shared.

Ghost / Halo Cells

Read-only copy of a neighbour's boundary, needed because the 3×3 kernel reads one row/col out. Exchange (1) a-priori or (2) overlapped with compute.

Surface vs Volume

compute $\propto$ **volume**; communication $\propto$ **surface**.

equal volume $N^2/4$: square surface $2N$ vs strip $\frac{5N}{2} \Rightarrow$ **square wins**.

10×10 : square 25 calc/20 transfers; strip 20/25.

Recipe: Surface/Volume Comparison

Volume = cells (compute); Surface = boundary cells (comms). Lower surface/volume wins for equal volume.

Ex. S12/13 – Square vs rectangle

For a 10×10 matrix in 4 chunks, which shape is better and why?

Equal volume, but square surface $2N <$ strip $\frac{5N}{2} \Rightarrow$ **square** = more compute per unit communication.

Ex. S16/17 – Apply to edge detection

Task parallelism or geometric? Chunks? Data sharing?

Geometric Decomposition; square tiles (or strips); share a 1-pixel **read-only ghost halo**; interior embarrassingly parallel.

Dependency & Mapping

Distributed Array

More chunks than UEs, assigned block-cyclic / round-robin so each UE owns several scattered chunks $\rightarrow$ smoother load balance.

Ex. S10 – Group & order chunks

How to group and order chunk tasks?

Group tasks on the same chunk; order = front pipeline, then **ghost-exchange before boundary**, then sobel_x/y $\rightarrow$ sobel_all join per chunk.

SPMD

SPMD

Single Program Multiple Data: one program on every UE, each with a unique ID/rank used to branch and index its data.

Recipe: SPMD Program

Init $\rightarrow$ get ID/rank $\rightarrow$ common code (branch/index by ID) $\rightarrow$ distribute data $\rightarrow$ collect+combine $\rightarrow$ finalise.

Ex. S27/28 – SPMD evaluated

Strengths and limits?

One maintainable, portable code base; overhead at start-up/shutdown; aligns with message passing. **Limit:** branching into genuinely different per-UE tasks stretches the pattern.

L3 – OpenMP

OpenMP Model

What OpenMP Is

Runtime library + #pragma directives + env vars. Shared-memory, Fork-Join + SPMD.

- **implicit barrier** at end of every parallel region

Parallel Region

#pragma omp parallel spawns a team of threads that all execute the following block.

Worksharing & Data Scope

parallel vs parallel for

- parallel alone: each thread runs the **whole** block (replicate)
- parallel for: iterations **split** across the team (worksharing)

Data Scope Clauses

- shared: one copy (concurrent writes = race)
- private: per-thread, uninitialised, discarded
- firstprivate/lastprivate: seed in / copy last out
- reduction(op:v): private partials combined (race-free)

Recipe: Predict OpenMP Behaviour

Read directives

replicate vs worksharing.

Scope each var

shared / private / reduction.

Find races

shared var written by > 1 iteration → indeterminate.

Count

multiply by thread count only in replicated regions.

Ex. S7 – How many prints?

T threads, *n* = 5: parallel alone vs parallel for?

T × 5 (replicated) vs 5 (shared). parallel makes threads; for splits work.

Ex. S8 – shared vs private value of b

shared(b) loop then private(b) loop, both b=a+i. Final b?

S1 shared: **race** → any of 7..16. S2 private: copies discarded → global unchanged. Final = whatever S1 left.

Ex. S21 – Race-free sum

How to sum an array without a race?

#pragma omp parallel for reduction(+:sum) – each thread keeps a private partial, combined after the join.

Synchronisation

Primitives

- critical: one block at a time
- atomic: single indivisible op
- barrier: phase wall
- sem_*, pthread_mutex_*

Barrier

- spin vs block (OMP_WAIT_POLICY)
- release order = OS, nondeterministic
- single nowait drops the barrier

Recipe: Pick a Sync Primitive

one block → critical; one op → atomic; accumulate → reduction; phase wall → barrier; cross-process → semaphore.

Ex. S18 – Barrier internals

Spin or block on arrival? Who runs first on release?

Depends: short + spare cores → spin; long/oversubscribed → block. Release order = **OS, nondeterministic** (use ordered).

Ex. S22 – nowait

Effect of nowait on single?

single = one thread runs the block (implicit barrier). nowait **removes the barrier** → others proceed (safe only with no dependency).

Scheduling & Runtime

Loop Schedules

- static: chunks up-front (even loads)
- dynamic: grab from queue (uneven)
- guided: shrinking chunks

Ex. S28 – Thread count

What determines the number of threads?

Precedence: if → num_threads → omp_set_num_threads → OMP_NUM_THREADS → default (≈#CPUs).

Flynn’s Taxonomy

The Four Classes

instruction streams × data streams:

- **SISD**: 1 ALU (single core)
- **SIMD**: 1 instr, many lanes
- **MISD**: lockstep / dependability
- **MIMD**: independent cores

SIMD → SIMT → SPMD

Increasingly abstract versions of the same lockstep-on-data idea (hardware → Nvidia threads → software pattern).

Recipe: Classify with Flynn

Count instruction streams and data streams: 1/1 SISD, 1/many SIMD, many/1 MISD, many/many MIMD.

Ex. S6/9 – SPMD pattern

What is SPMD and why is it here?

One program per UE, unique ID, branch/index by ID. It is the software generalisation of **SIMD** (same instruction, different data).

ARM NEON

NEON Registers & Lanes

ARM SIMD unit.

- **Q** (128-bit) = 2×64 / 4×32 / 8×16 / 16×8
- **D** (64-bit) = half as many lanes

NEON Properties

- built around **MAC** (multiply-accumulate)
- **no divide** (reciprocal estimate only)
- type name <base><bits>x<count>_t

Recipe: Decode a NEON Intrinsic

Pattern v <op> q? _ <type>:

op

the operation (mul, mla, ...).

q?

present ⇒ 128-bit Q, else 64-bit D.

type

<base><bits>x<count>: **count** = lanes, **bits** = lane width.

result

lane-wise op over **count** elements.

Ex. S18/19 – Decode intrinsics

vmulq_f64? lanes in uint16x4_t? vmul_lane_u16(a,b,L)?

vmulq_f64: 128-bit lane-wise multiply of **2 doubles** → float64x2_t. uint16x4_t: **4 lanes**. vmul_lane: broadcast b[L] to all lanes, multiply by a.

Ex. S20 – Lab: lanes/iterations

Process *K* elements with NEON – how many vector ops?

lanes = vector width / element size; iterations = ⌈*K*/lanes⌉. E.g. uint16x8 → ⌈*K*/8⌉ vector ops (+ MAC for the filter).

MISD / Lockstep

Dependable Computing

MISD spends **redundant hardware** on reliability, not speed (opposite goal to SIMD/MIMD).

Lockstep Variants

- tight: 2 CPUs 0–2 clk apart, compare bus → RESET (e.g. ABS)
- loose: async, dissimilar impls + acceptance voting
- **2oo3**: one fault tolerated

Ex. S21–24 – Real-world MISD

Where do you see lockstep in practice?

U96 / Zynq ZU3EG, 2×Cortex-R5F in tightly-coupled lockstep.

Multiprocessor Scheduling

SQMS vs MQMS

- **SQMS**: 1 shared queue – simple, but lock contention + cache bounce
- **MQMS**: per-CPU queue – cache affinity, but load imbalance

Balancing

- **cache affinity**: keep a task on its CPU
- **work-stealing**: idle queue takes work from a busy one

Ex. S6 – Explain SQMS/MQMS
Trade-offs and how to fix imbalance?

SQMS = balanced but contended; MQMS = affinity but imbalanced; fix via migration / **work-stealing**.

Scheduler Categories

Linux Policies

- **RT**: FIFO (run-to-completion), RR (time-sliced), DEADLINE (EDF)
- **Fair**: OTHER = EEVDF, RB-tree on vruntime, tuned by nice
- **Low**: BATCH, IDLE

Recipe: Pick a Scheduler

hard deadline → DEADLINE; strict priority → FIFO/RR; fairness → OTHER; background → BATCH/IDLE.

Ex. S7-9 – Three categories
Name the scheduler categories and their goals.

RT (strict priority, N top run), Fair (utilisation/fairness via EEVDF), Low (background). DEADLINE can bypass balancing.

Amdahl's Law

Amdahl's Law
fraction p parallelisable on N processors:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}, \quad S_{\max} = \frac{1}{1 - p}$$

Real $S_{\max}$ limited by coherence + sync overhead.

Recipe: Apply Amdahl's Law

Find p → plug N into $S(N)$ → ceiling $S_{\max} = \frac{1}{1 - p}$.

Ex. S8 – Worked
 $p = 0.75$: speed-up on 4 cores? maximum?

$$S(4) = \frac{1}{0.4375} \approx 2.29\times; S_{\max} = \frac{1}{0.25} = 4\times.$$

Synchronisation

Critical Section

RW-shared data; one task inside at a time. Atomicity via: disable IRQ, ISA atomic, Peterson/spinlock, language, OS mutex/semaphore (**blocks, no spin**).

TAS vs TTAS

- **TAS**: atomic XCHG, spins with writes → bus lock + invalidation
- **TTAS**: spin on cached read first, write only when free → near-ideal

Recipe: Build a Lock

atomic op (TAS) → cheaper TTAS (read first) → or block via OS mutex/semaphore.

Ex. S11 – TAS vs TTAS
Why is TTAS faster under contention?

TAS spins with atomic writes (bus lock + invalidation); TTAS spins on a cached read, writes only when the lock looks free → far less coherence traffic.

Cache & Paging

Cache

Fast memory near/in the CPU, **transparent** to the programmer.

- hit → served fast
- miss → fetch a whole **line** from memory (also cached)

Why a Hierarchy

The CPU is far faster than RAM (the bottleneck), so caches + paging hide latency and use main memory efficiently.

Ex. S4/5 – Explain cache & paging

What is a cache (hit/miss)? What is paging (resident vs working set)?

Cache = fast transparent memory; miss → fetch a line. Paging = pages→frames, only used pages resident; **resident** = in memory, **working** = in use now.

The Memory Access Walk

Access Chain

CPU pipeline (IF-ID-EX-MEM-WB) → Cache → **MPU** (rights) → **MMU** (TLB = recent mappings; else page-table walk) → **page fault** → MM software.

Recipe: Trace a Memory Access

Pipeline

stage requests an address.

Cache

hit → done; miss → continue.

MPU

access allowed?

MMU/TLB

hit → frame; miss → walk page tables.

Page fault

empty → MM loads frame from disk, updates page dir.

Fill

TLB caches mapping; cache reads a line; write sets **dirty bit**.

Ex. S6–12 – Worked code trace

Walk mov ea,var1; mov eb,var2; add ea,eb; mov var3,ea. Why first fetch slow, next fast?

First fetch (0x100): cache miss → MPU → MMU → TLB miss → walk empty → **page fault** (load frame #2). Fill = whole line (I1–I4) + TLB mapping ⇒ 0x104,0x108 are **cache hits**. var1 (0x400): 2nd page fault (frame #8); var2 = hit; mov var3 = write → **dirty bit**.

Thrashing & Context-Switch Cost

Thrashing

- cache**: many lines map to one slot → evict/miss loop
- disk**: too few frames → page evict/reload loop

Paging

Address space → pages (4K/2M); physical → frames.

- load only used pages (any frame)
- resident set** = pages in memory
- working set** = pages in use now

Switch Cost

- cache **high** (flush + write-back)
- TLB **high** (flush)
- MPU/page-tables medium; pipeline low

Ex. S13–15 – Thrashing & switch cost

Explain both thrashing types and the cost of cache-oblivious code.

Cache thrash = repeated misses (same slot); disk thrash = repeated page evict/reload. Switch cost dominated by cache+TLB flush; CPI spikes then re-warms.

Cache Writeback & Multicore

Writeback (dirty bit)

A dirty line is written back when (1) the controller decides, or (2) it needs the line (eviction). Then dirty→0; eviction sets valid→0, refill, valid→1.

Recipe: Will a Writeback Happen?

Check the dirty bit: dirty + (controller decision or eviction) → write back first. Clean line → just refill.

Ex. S16–18 – Writeback & multicore L1/L2

When does a dirty line write back? L1 read/write-back on shared L2?

See recipe. Multicore: read pulls L2→L1; **flushing L1 requires flushing L2**; L1→L2 write-back may trigger L2→memory; coherence keeps other L1s consistent.

Cache Coherence

Coherence

Visibility of a write across cores. A write **will** become visible (order to one location preserved) – says nothing about **when**; not in ISA; necessary not sufficient.

Snooping

Bus broadcast; controllers listen, invalidate/update. Non-scalable.

False Sharing

Two cores write **different** vars in the **same line** → line ping-pongs (useless coherence traffic). Fix: pad/align to separate lines.

Recipe: Diagnose Coherence / False Sharing

Same line in ≥2 caches + a write → invalidation traffic. Same var = true sharing; different vars = **false sharing** → pad/align.

Ex. S19/20 – Coherence vs consistency

Define both; what does 0-1-1-2 / 0-2-1-2 show?

Two cores observe writes in **different** order → a protocol must make P2 see P1's order (coherence). Consistency = cross-location ordering (sequential/processor/release).

Ex. S21–23 – Protocols & false sharing

Outline snooping vs directory; explain false sharing.

Snooping = bus broadcast invalidation (e.g. P1 writes var1 → P2 invalidated), non-scalable. Directory = owner-based, scalable. False sharing = different vars, same line → ping-pong; fix by padding.

Consistency

Ordering of accesses to **different** locations as seen by another core.

- sequential / processor / release
- release: order only at barriers

Directory

Point-to-point; owner invalidates sharers, waits ack, writes, becomes owner. Scalable.

Ex. S24 – ASID / CPU pinning

How is cache/TLB state preserved across switches?

Physical-address tagging or an **ASID** lets one process's lines survive while another runs ⇒ cheap to re-schedule on the same core (**CPU pinning / affinity**).

Memory Architectures

UMA / NUMA / SUMA

- **UMA**: equal access, ~4-CPU limit, L2/L3 buffer
- **NUMA**: fast local, slow remote via interconnect (ccNUMA)
- **SUMA**: interleaving → best-case UMA

Recipe: Classify a Memory Architecture

equal access → UMA; fast-local/slow-remote → NUMA; interleaved-to-fake-contiguous → SUMA.

Ex. S25–29 – Architectures & OS support

Bus limits? UMA/NUMA/SUMA? How does Linux handle NUMA?

Bus latency grows with length → ~4-CPU limit. UMA/NUMA/SUMA as above. Linux maps HW **cells** → **nodes**, keeps work+memory local (affinity), **zonelists** by NUMA distance; `taskset/numactl/sched_setaffinity`.

L7 – CPU Architectures

Memory Architectures

Three Architectures

- **Von Neumann**: shared I+D bus
- **Harvard**: separate I/D
- **Modified Harvard**: split L1, unified L2/RAM

Parallelising a CPU

- multicore
- multiple FUs (data parallelism)
- multiple pipelines (instruction)
- heterogeneous PEs

Recipe: Classify a CPU Architecture

shared bus → Von Neumann; fully separate → Harvard; split L1 + unified L2 → **Modified Harvard** (e.g. A53).

Ex. S4–7 – A53 architecture

Separate L1-I/L1-D, unified L2 – which architecture? Ways to parallelise?

Modified Harvard (Harvard inside, Von Neumann outside). Parallelise via multicore, FUs, pipelines, PEs.

Multiple Functional Units (VLIW)

VLIW

The **compiler** packs one op per FU (.M/.D/.L/.S) into a wide instruction word, run in parallel. Big code, binary-incompatible; + software pipelining.

Ex. S8–10 – DSP

What memory architecture? How does execution work?

Harvard (separate program/data bus keeps MAC fed); **VLIW** execution + software pipelining.

Pipelines

Hazards

- structural, data, control
- RISC = pipeline-friendly; CISC = microcoded

Pipeline & Issue

- super-pipeline: finer stages (faster)
- superscalar: more pipelines (several/cycle)
- order: in-order → OoO exec → OoO issue+exec

Recipe: Identify the Parallelism Mechanism

multiple FUs (VLIW)=data; multiple pipelines (superscalar)=instruction; idle slots filled by 2nd thread=SMT; separate cores=multicore/AMP.

Ex. S11–14 – Pipelines

Super-pipeline vs superscalar? Issue/execute order?

Super-pipeline = finer stages (issue faster); superscalar = more pipelines (several/cycle). Order: in-order → OoO exec → **OoO issue+exec**.

Co-processors & SMT

Co-processor

CPU passes unrecognised instructions to it and **stalls** (8087 + 8086 on FMUL); own ISA/clock/mem (NEON). Costly decoders.

SMT (Hyperthreading)

2 threads/core fill **idle issue slots**. Requires a **superscalar** core (spare slots). ~30% gain; security issues.

Ex. S15–19 – Co-processor & why SMT needs superscalar

How does a co-processor work? Why does SMT need superscalar?

Co-pro: CPU offloads + stalls. SMT fills idle slots with a 2nd thread; only superscalar cores have multiple slots/cycle to fill – a scalar core has nothing spare.

Asymmetric Multiprocessing

AMP Scenarios

different/same ISA, different clocks, big.LITTLE (same ISA), different/no OS.

openAMP

- remoteproc: boots a remote core's ELF
- rpmsg over virtio: ring-buffer messaging

Recipe: Set Up AMP

same ISA → SMP/big.LITTLE one OS; different ISA → separate OS/bare-metal per core + openAMP (remoteproc, rpmsg/virtio).

Ex. S20–25 – Different ISA under one OS?

Can different-ISA cores share one OS? How do they communicate?

No (one kernel = one ISA; big.LITTLE works only because same ISA). Different ISA → separate OSes + openAMP. Zynq: 4×A53 SMP + 2×R5 lockstep + FPGA.

GPU vs CPU & the CUDA Model

GPU vs CPU

- CPU: latency-optimised (few fast threads)
- GPU: **throughput** (many slow threads)

CUDA Hierarchy

- kernel → thread (1/element)
- threads → block → grid
- block → **SM**; SM runs **warps of 32**

SM (Streaming Multiprocessor)

HW unit a block runs on. Fermi SM: 32 cores, 4 SFU, 16 L/S, 64KB shared/L1, warp scheduler. Grid of blocks distributed across SMs.

Recipe: Size a CUDA / OpenCL Launch

One thread per element

#threads = #elements.

Block/grid

threadsPerBlock multiple of 32; grid = ⌈elements/block⌋.

Index ranges

[R] [C] → threadIdx.x ∈ 0..R-1, .y ∈ 0..C-1.

Ex. S4-7 – GPU vs CPU & thread count

How do GPU/CPU differ? Threads for a [10][5] op? threadID range?

CPU latency- vs GPU throughput-optimised. **10 × 5 = 50 threads**; threadIdx.x ∈ 0..9, .y ∈ 0..4 (IDs (0,0)..(9,4)).

Ex. S8-11 – SM & limitations

What is an SM? Why not one fine thread for everything?

SM = unit a block runs on (warps of 32). Limits: branch divergence serialises a warp, memory coalescing matters, too-fine work wastes scheduling.

Rendering Pipeline

Stages

world coords → vertex shader → assembly → (geometry) → rasterise → fragment shader (per pixel) → blend.
Mali: Utgard (tiling) → Midgard (OpenCL).

Ex. S14/15 – Why so parallel?

Why is rendering massively parallel?

Data **expands** from a few vertices to millions of pixels → per-pixel work is independent and huge.

OpenCL

Four Models

- Platform (Host→Device→CU→PE)
- Execution (work-item/group/NDRange)
- Memory (weak consistency)
- Programming

Memory Regions

- Global, Constant
- Local (per work-group)
- Private (per work-item)

Recipe: Convert a Loop to a Kernel

drop the loop → i=get_global_id(0) → body on element i; work-items independent.

OpenCL Vector Add

```
1 __kernel void vadd(__global float* a,
2                   __global float* b,
3                   __global float* c) {
4     int i = get_global_id(0);
5     c[i] = a[i] + b[i];    // one work-item per element
6 }
```

Ex. S27/28 – Loop → kernel

Rewrite for(i<4096) c[i]=a[i]+b[i]; and give the NDRange.

See code box. NDRange = 4096 work-items (e.g. group 512); the loop is replaced by the index space.

Ex. S21-26/31-33 – Models & sync

Name the four models; where can you synchronise?

Platform/Execution/Memory/Programming. Sync **only within a work-group** (barrier, fence, atomics); host enqueues in order, completion in/out-of-order.

GPU Scheduling

Coarse vs Fine

- coarse: blocks → SMs
- fine: warp scheduler interleaves 32-thread warps to **hide latency**

Ex. S29 – Why interleave warps?

Why does the warp scheduler interleave?

When one warp stalls on memory, another ready warp runs → execution units stay busy (latency hiding).

AMP Recap

Ex. S40/41 – Different ISA under one OS?

No – separate OSes coupled by openAMP (remoteproc + rpmsg/virtio). Same as L7.

Distributed Architectures

Tight vs Loose Binding

- tight: shared memory over interconnect
 - **loose**: distributed memory (MPI's home, SPMD→MIMD)
- Protocols rise: variable → scatter-gather → RPC → distributed objects.

Ex. S2/6 – Why MPI?
What kind of system does MPI target?

Distributed memory: independent processes cooperate only by messages; communication is synchronisation.

MPI Basics

Six Core Ops

- Init, Finalize
- Comm_size, Comm_rank
- Send, Recv

Recipe: Write an MPI Program
Init → Comm_size/Comm_rank → branch by rank → Send/Recv or collectives → Finalize. Build mpicc, run mpirun -n X.

Ex. S7–13 – Basics & launch
Six ops? Communicator/rank? SPMD vs MIMD launch?

See defs. SPMD: mpirun -n X ./p (X>cores = oversubscribe). MIMD: mpirun -n 2 p1 : -n 2 p2. Bind -cpu-set.

Blocking vs Non-blocking

- Send/Recv: block until buffer empty / data in
- Isend/Irecv: return immediately → Wait/Test (overlap)

Four Send Modes

- **Standard** (Send)
- **Synchronous** (Ssend) – safest, waits for recv
- **Buffered** (Bsend) – app buffer, decoupled
- **Ready** (Rsend) – lowest overhead, recv pre-posted

Communicator & Rank

MPI_COMM_WORLD groups processes; each has a unique **rank** used (SPMD) to branch and pick its data.

Recipe: Choose a Send Mode

rendezvous → Ssend; decouple → Bsend; recv posted → Rsend; else Send; overlap → Isend/Irecv.

Ex. S14–16 – Blocking & modes
Blocking vs non-blocking? Which send guarantees the receiver started?

Blocking waits on the buffer; non-blocking returns + Wait/Test. **Ssend** completes only once the matching receive begins.

Messages & Datatypes

Message Anatomy

Envelope (source, dest, communicator, tag) + **Body** (count, datatype, buffer). Derived types via MPI_Type_create_struct.

Ex. S17–19 – Message structure
What's in an MPI message? Why standard datatypes?

Envelope + body. Standard types (MPI_INT/FLOAT/...) so both ends interpret bytes identically across heterogeneous nodes.

Collectives & Reduction

Collectives

- Bcast: root→all (tree)
- Scatter/Gather: chunks out/back
- Reduce/Allreduce: combine
- Barrier

Recipe: Distributed Reduction
each process holds a partial → pick op (SUM/MAX/MIN/...) → MPI_Reduce(send,recv,count,type,op,root,comm) (element-wise); Allreduce → all.

Ex. S20/21 – Collectives & reduction
Explain bcast/scatter/gather. Where have we seen reduction?

Bcast = root→all; scatter = out; gather = back. Reduction: OpenMP reduction (L3) + accumulate (L1-2); **MPI_Reduce** = distributed version, element-wise across procs.

Cross-lecture threads: Reduction OpenMP(L3)↔accumulate(L1-2)↔MPI_Reduce(L9) · SPMD pattern(L2)←SIMD/SIMT(L4)→MPI(L9)→CUDA/OpenCL(L8) · Scheduling L1→L3→L5→L8 · Affinity MQMS(L5)↔ASID(L6)↔-cpu-set(L9) · Amdahl(L5) caps all.